INDIAN PEDIATRICS
AI ASSISTANT

STEP 5 | APPLICATION

A local interface for search, evidence and comparison

This chapter explains how the first user-facing application presents hybrid journal search results, source provenance, article comparison and metadata filters, with optional answer synthesis behind the explicit processing-consent gate.

Teaching manual installment 1.0 | October 2026

Project: Reusable medical-journal literature assistant
POC corpus: Indian Pediatrics, January 2025, Volume 62, Issue 1
Audience: AI students and developers learning to plan and build a grounded literature assistant

How to use this installment
These are the first chapters of a cumulative teaching manual. They explain decisions made so far and provide a reproducible way to understand them. The full 300+ page reference is intended to grow as later roadmap phases are completed.

Chapter map

- 26. Streamlit UI - create a simple Windows-friendly local interface.
- 27. Search - connect the question and metadata filters to the local index.
- 28. Answer - present gated, structured, citation-checked synthesis or an explicit fallback.
- 29. Sources - expose article, page, source file and publisher traceability.
- 30. Compare - place retrieved passages from selected articles side by side.
- 31. Filter by article type - constrain retrieval to selected journal sections.

Learning objectives

- Run the application locally from the Project folder.
- Explain which logic belongs in the interface and which belongs in shared retrieval modules.
- Distinguish a retrieved passage from an answer synthesized by a language model.
- Trace an evidence card back to issue metadata and a local source PDF.
- Configure the same interface for a future journal without hard-coding its name or corpus directory.

26. Streamlit UI

Step 5 places a small local interface over the Step 4 retrieval layer. It is a demonstration front end, not the final packaged Windows desktop product. Streamlit runs a local application server and opens its interface in a browser window; its documented launch command is `streamlit run your_script.py`.

26.1 What, why, and how

- What: a search page with question input, article-type/year/author filters, source cards, side-by-side comparison, and a visible safety boundary.
- Why: editors and students can see the evidence trail and workflow before investing in a fully native or packaged desktop interface.
- How: `app/streamlit_app.py` loads the journal profile and calls shared configuration, retrieval and answer-boundary code. `scripts/run_ui.py` launches the local Streamlit server using the active project Python environment.

26.2 Why this interface first

Option	Why it fits this POC	What to revisit later
Streamlit local UI	Fast to build, easy to inspect, and Python-native for this one-issue demonstration.	Packaging, multi-window behavior, polished Windows installation and offline updates.
Native Windows UI	Could feel like a conventional installed desktop program.	Would add UI and packaging work before the retrieval workflow is evaluated.
Web front end with API backend	Good separation for a deployed service or multi-user product.	Adds a separate service and deployment boundary not needed for the local demo.

26.3 Page structure and state

- The journal name, corpus title, metadata file and source-PDF folder come from the selected journal profile.
- The sidebar holds article type, year, author and result-count controls; the main panel holds the question, answer status, sources and comparison.
- A search action stores its retrieved evidence and status in session state so that Streamlit reruns do not discard the current result.

- The optional Groq provider is called only when the local external-processing consent gate is enabled, credentials are present, evidence was retrieved, and the question is not patient-specific.

27. Search

Search turns a question and optional metadata constraints into candidates from the local index. The current configured index combines local lexical full-text and FastEmbed semantic retrieval, then uses reciprocal-rank fusion to order results.

27.1 What, why, and how

- What: pass the question, selected section types, publication year, author text, allowed content roles, query embedding and desired result count to the shared retrieval module.
- Why: a focused query and filters help the reader find specific published material and limit unrelated issue content.
- How: the UI loads `config/rag.toml`, embeds the query locally, calls `retrieval.search` for hybrid ranking, and renders the ranked passages. Groq is used separately for optional answer synthesis.

27.2 Search behavior and limits

- Lexical and vector results are combined; neither a high semantic match nor a lexical match by itself proves that a passage answers the question.
- Author and year filters constrain metadata. Article types are mapped to the stored journal section/category.
- The current corpus is one issue from 2025, so a later-year filter will naturally return no records.
- The model can abstain when retrieved context does not answer the question, but its abstention and grounding still require evaluation.
- The semantic similarity cutoff is an initial POC setting, calibrated only against the current small Golden Test Set; it must be retuned on a broader corpus.

27.3 Interface-to-backend boundary

The Streamlit file should collect input, call stable shared functions, and display results. It should not implement its own ranking formula, citation rules, PDF parser, or provider client. Keeping those decisions in `src/journal_assistant/rag/` makes a future desktop or web front end reuse the same behavior.

28. Answer

The answer area is status-aware. When configured, the gated Groq provider receives the question and retrieved passages, and returns structured claims with source IDs. The application verifies those IDs against retrieved evidence and adds visible citations from its own source records.

28.1 What, why, and how

- What: show generated, no-evidence, patient-specific boundary, unavailable-provider, or generation-error states.
- Why: users must distinguish source discovery from AI synthesis and see when the model abstains.
- How: `answer_question` blocks patient-specific questions, returns a no-evidence state when no passages are found, parses the provider's structured output, and rejects empty or invalid source IDs. The UI renders source cards using the original retrieved records.

Current answer status
The configured Groq path has passed a basic structured-citation example and an out-of-corpus abstention example. This is a limited technical validation, not clinical validation or a substitute for human review.

28.2 Safety presentation

- Keep the research-literature-only warning visible near the question input.
- A narrow patient-specific phrase detector is only a prototype safeguard; it is not a complete safety classifier.
- Do not turn a search result into treatment advice or imply that one paper establishes a universal clinical recommendation.
- The application verifies that cited source IDs were retrieved. A human must still verify that each passage substantively supports its claim.

29. Sources

Source cards make the evidence trail visible. Each card uses fields attached to its chunk, rather than asking a model to invent a bibliographic reference.

29.1 What, why, and how

- What: display source label, article title, authors when verified, year, category, volume, issue, journal page, physical PDF page, source filename and abbreviated source checksum.
- Why: a reader needs to inspect the actual article passage and see which page and file it came from.
- How: source labels are assigned after retrieval. The UI links to the publisher issue page and offers a local source-PDF download when that file exists in the configured source directory.

29.2 Citation traceability and limits

- A source label points to a retrieved chunk, not automatically to claim-level proof.
- Printed journal page and physical PDF page are shown separately because they may differ.
- The source checksum is a short display of the full SHA-256 retained in metadata; it helps distinguish the corpus file used.
- The link currently opens the issue landing page. A later interface can add direct PDF page navigation after confirming stable publisher URLs and permission.
- Keep original source PDFs unchanged; the download button serves the configured local copy.

30. Compare

The comparison panel lets a reader inspect two or more retrieved articles beside one another. This is a passage comparison interface, not a generated synthesis of study findings.

30.1 What, why, and how

- What: choose articles present in the current result set and view their article type, authors, year, article ID and matching passages in columns.
- Why: researchers often need to compare related publications, methods or evidence types, and seeing each passage separately preserves its context.
- How: the UI groups retrieved chunks by article ID and displays up to three selected articles side by side with their source labels.

30.2 Avoiding false comparison

- Do not claim one study is superior based only on lexical matches or an excerpt.
- Do not treat two passages as methodologically comparable until design, population, outcomes and limitations are reviewed in the full articles.

- Do not hide disagreement or limitations. A future synthesis workflow must cite each claim to the relevant source and be tested against reviewed examples.
- For the current POC, the reader performs the interpretation; the interface only organizes retrieved evidence.

31. Filter by article type

Article type or section filters let users narrow retrieval to the journal's editorial categories, such as Original Article, Systematic Review, Medical Education, or Correspondence.

31.1 What, why, and how

- What: a multi-select control populated from the current journal metadata catalogue.
- Why: category can change how a paper should be interpreted and which material is useful for a research question.
- How: selected values are passed as a set of exact section filters; the local store excludes rows whose section is not selected.

31.2 Data requirements and limitations

- A category filter is only as accurate as the article metadata file; the UI cannot correct a misclassified record by itself.
- Category labels may differ between journals. Keep original journal labels and use a per-journal taxonomy mapping if normalized labels are added later.
- An empty multi-select means all eligible default roles; it does not remove role-based exclusions for administrative or advertisement records.
- Use the article title and source card to confirm a selected result's category before citing it.

Reusable setup for another journal

To adapt the UI, copy the journal template, set its display name and initial corpus details, and point the `[corpus]` fields to that journal's metadata CSV and raw source folder. The metadata must follow the shared schema or an explicit journal adapter must map it. The application reads `JOURNAL_ASSISTANT_CONFIG`; it does not infer or download a new publisher's corpus automatically.

> **Permission is journal-specific**
> A local interface does not create permission to collect, index, or share another publisher's articles. Record and review access terms for each journal before using its source material.

Checkpoint and troubleshooting

`scripts/check_step5.py` runs an in-process Streamlit smoke test and exercises a known query, article-type filter, no-result question, and patient-specific boundary. It makes no provider calls and writes `data/processed/step5_checkpoint.md`.

Symptom	Likely cause	Action
Streamlit command not found	Virtual environment is inactive or dependency is missing.	Activate `.venv` and run the pinned requirements installation.
The index is not built	Step 4 local index is absent.	Complete the local Step 4 chunk/index commands; do not enable external processing just to run the UI.
No passages appear	Query terms or metadata filters do not match this one-issue index.	Remove optional filters, try article title terms, and inspect the Step 4 checkpoint.
No generated answer	Expected while provider-backed RAG remains unconfigured.	Review the displayed source passages; Step 4 provider setup and validation are still required.

Symptom	Likely cause	Action
Wrong article type appears	Metadata field is inaccurate or category values differ.	Correct the controlled metadata record and rerun its quality checkpoint.
Source PDF download is missing	Local raw file is absent or a source filename is wrong.	Check the journal profile source folder and file manifest; preserve the original issue file.

Step 5 checkpoint outcome

Application checks passed
The Step 5 UI smoke check covers app rendering, local hybrid retrieval, article-type filtering, no-result handling, and the patient-specific boundary. Provider-backed generation is conditionally available when the local consent switch and credentials are present. The Step 6 evaluation records the limited live answer checks.

A green Step 5 UI check means the local application workflows render and basic interactions run. It does not imply clinical validation, complete journal coverage, or permission for external sharing.

References and glossary

- Ver 1 - Complete Implementation Guide for POC.pdf, Phase 5 Application and activities 26-31.
- Ver 1 - Master Guide for POC.pdf, user interface and quality-assurance principles.
- Streamlit official run instructions: `https://docs.streamlit.io/develop/concepts/architecture/run-your-app`.
- Project files: `app/streamlit_app.py`, `scripts/run_ui.py`, `scripts/check_step5.py`, `config/journals/`, and `data/processed/step5_checkpoint.md`.

Glossary

Term	Plain-language meaning
Session state	Values Streamlit retains across a user's interactions and reruns.
Lexical search	Search based on words and terms present in indexed text.
Source card	A UI item showing a retrieved passage and its verified provenance fields.
Article type	The editorial section/category assigned in the journal metadata.
Local application server	A program serving the UI on the same computer for the POC.
Extractive display	Showing retrieved source text without claiming it is an AI-generated summary.